

Array-based lists

Array-based implementations

of the **List** and **Queue** interfaces

	get(i) / set(i,x)	add(i,x) / remove(i)
ArrayStack	$O(1)$	$O(1 + n - i)$
ArrayDeque	$O(1)$	$O(1 + \min\{i, n - i\})$
DualArrayDeque	$O(1)$	$O(1 + \min\{i, n - i\})$
RootishArrayStack	$O(1)$	$O(1 + n - i)$

Interface - List

List – represents an indexed sequence of elements

size()	– returns the number of elements on the list (n)	}	$O(1)$
isEmpty()	– returns whether list is empty		
get(i)	– returns the element at position i		
set(i, x)	– update the element at position i to be x		
add(i, x)	– add the element x to position i	}	$O(n - i + 1)$ amortized time
remove(i)	– remove element at position i		

The Java Collections Framework documentation about List interface:

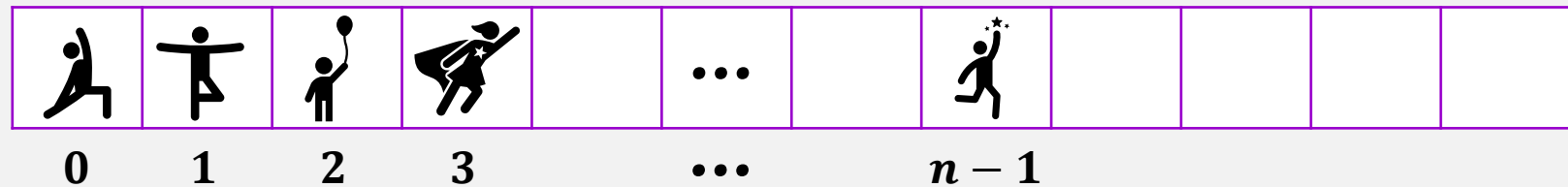
<https://docs.oracle.com/javase/8/docs/api/java/util/List.html>

ArrayStack (aka ArrayList)

ArrayStack is an array-based List implementation. It is equivalent to the **ArrayList** in the Java Collections Framework (JCF). It has the same performance.

Array a of type T and
of size $a.length$

$T[]$ a



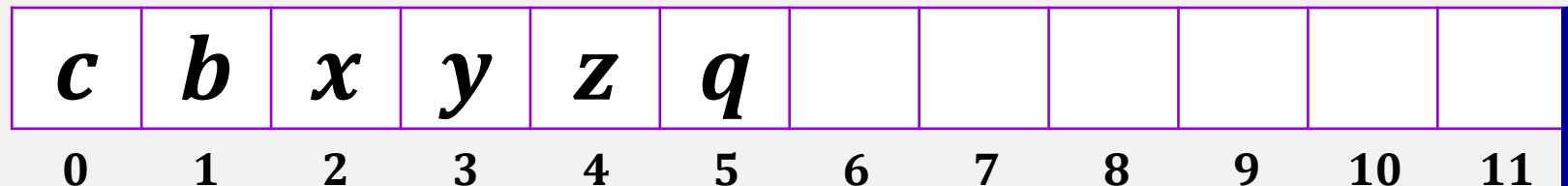
int n



size of the list (can be different from $a.length$)

char[] a

$n = 6$



$a.length$
 $= 12$

ArrayStack

Constructor `ArrayStack()`:
 $a = \text{new } T[1];$
 $n = 0;$



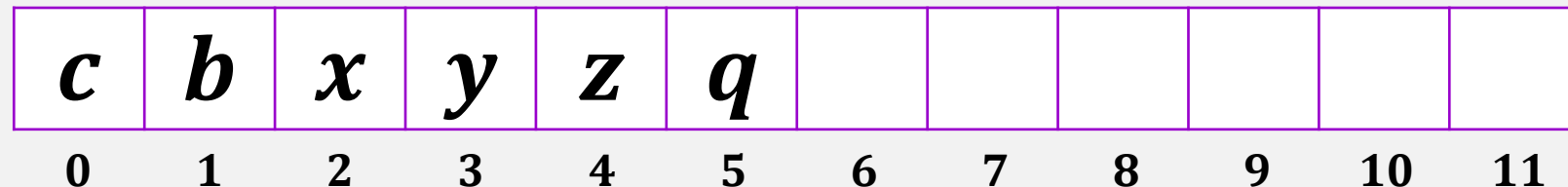
returns the
element at
position i

`T get( $i$ ):`
if ($i < 0 \parallel i \geq n$) then throw new `IndexException`;
return $a[i]$;

notice, not $i > a.length$

$O(1)$

`char[] a`
 $n = 6$



$a.length$
 $= 12$

ArrayStack

- saves the element at position i ;
- updates the element at position i to be x ;
- returns the element that was at position i

T **set**(i, x):

if ($i < 0 \parallel i \geq n$) then throw new IndexException;
T $y = a[i]$;
 $a[i] = x$;
return y ;

$O(1)$

set(4, w):

char[] a

$n = 6$

<i>c</i>	<i>b</i>	<i>x</i>	<i>y</i>	<i>w</i>	<i>q</i>						
0	1	2	3	4	5	6	7	8	9	10	11

$a.length = 12$

ArrayStack

returns the number
of elements on the
list (n)

```
int size():  
    return  $n$ ;
```

we do not return
 $a.length$

$O(1)$

char[] a
 $n = 6$

<i>c</i>	<i>b</i>	<i>x</i>	<i>y</i>	<i>w</i>	<i>q</i>						
0	1	2	3	4	5	6	7	8	9	10	11

$a.length$
 $= 12$

ArrayStack

- check bounds
- if n is equal to $a.length$ then resize the array a
- shift elements i through $n - 1$ up an index (to indices $i + 1, \dots, n$)
- insert the element x to position i
- increase n by 1 to reflect the update

void add(i, x):

i can be equal n

$O(1 + n - i)$

```

if ( $i < 0 \parallel i > n$ ) then throw IndexException;
if ( $n + 1 > a.length$ ) then resize();
for ( $j = n; j > i; j --$ ) }
     $a[j] = a[j - 1];$ 
 $a[i] = x;$ 
 $n ++;$ 
    
```

shift

we will see this function later

after a call to **resize()**, we can be sure that $a.length > n$.

add(2, h):

char[] a

$n =$ ~~6~~⁷

c	b	h	y	w	q						
0	1	2	3	4	5	6	7	8	9	10	11

$a.length = 12$

ArrayStack

- check bounds;
- save the element at position i ;
- shift elements $i + 1$ through $n - 1$ down an index (to indices $i, \dots, n - 2$);
- decrease n by 1;
- if a is 3 times longer than necessary then resize the array;
- return the element that was at position i .

$$O(1 + n - i)$$

T remove(i):

```
if ( $i < 0 \parallel i \geq n$ ) then throw IndexException;  
T  $x = a[i]$ ;  
for ( $j = i; j < n - 1; j++$ ) } shift  
     $a[j] = a[j + 1]$ ;  
 $n--$ ;  
 $a[n] = \text{null}$ ;  
if ( $3n \leq a.\text{length}$ ) then resize();  
return  $x$ ;
```

remove(3):

char[] a

$n =$ ~~7~~**6**

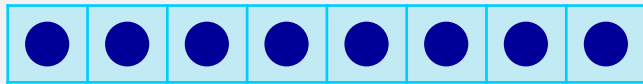
<i>c</i>	<i>b</i>	<i>h</i>	<i>x</i>	<i>y</i>	<i>w</i>	<i>q</i>						
0	1	2	3	4	5	6	7	8	9	10	11	

$a.\text{length}$
 $= 12$

ArrayStack – `resize()`

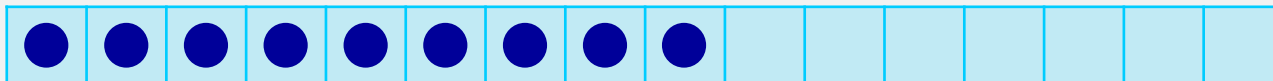
There are two situations that trigger `resize()`:

1. Array is full and we want to add another element

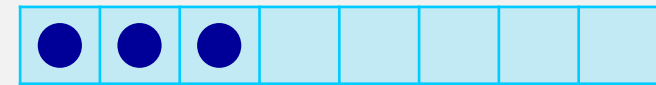


`add(i, x):`

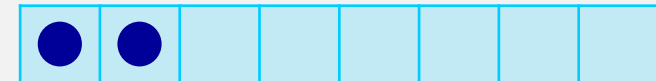
`resize():`



2. We remove an element, and the array becomes 2/3 empty



`remove(i):`



`resize():`



ArrayStack – resize()

void **resize()**:

$T[]$ ***b*** = new array(**$\max\{2n, 1\}$**)

for (***j*** = **0**; ***i*** < ***n***; ***i***++)

b[***i***] = ***a***[***i***];

a = ***b***;

executes ***n*** times

to avoid 0-length array

$O(n)$

- create new array ***b***;
- copy everything from ***a*** to ***b***;
- the new array becomes ***a***.

ArrayStack – `resize()`

```
void resize():
```

```
    T[] b = new array of size s  
    for (j = 0; i < n; i++)  
        b[i] = a[i];  
    a = b;
```

- take $s \gg n$
 - + you never have to **resize()** again (fast!)
 - wastes a lot of space
- take $s = n + 1$
 - + does not waste any space
 - wastes too much time in **resize()** over many calls
- take $s = \max\{2n, 1\}$
 - wastes $O(n)$ space
 - + gives us $O(1)$ amortized run time

Theorem 2.1

An **ArrayStack** implements the **List** interface. Ignoring the cost of calls to **resize()**, an **ArrayStack** supports the operations

- **get(i)** and **set(i, x)** in $O(1)$ time per operation; and
- **add(i, x)** and **remove(i)** in $O(1 + n - i)$ time per operation.

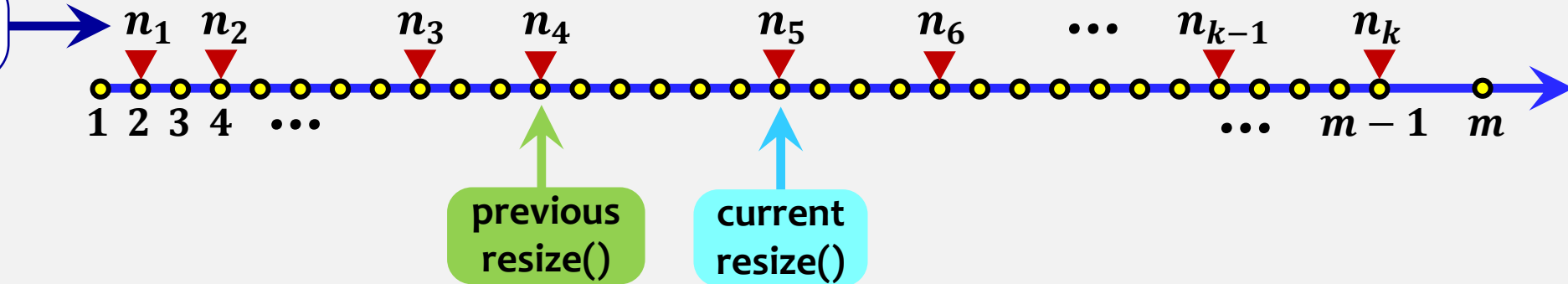
Furthermore, beginning with an empty **ArrayStack** and performing any sequence of m **add(i, x)** and **remove(i)** operations results in a total of $O(m)$ time spent during all calls to **resize()**.

Theorem 2.1 - proof

▼ `resize()`

● add/remove operation

of elements
in the list



The total # of elements copied by all the calls to `resize()` is at most

$$n_1 + n_2 + n_3 + \dots + n_k$$

Theorem 2.1 - proof

We resize in two cases:

- on **add**, when $n + 1 > a.\text{length}$ (when $a.\text{length} = n$)
- on **remove**, when $3n \leq a.\text{length}$

We resize to $\max\{2n, 1\}$ (i.e., $2n$ unless $n = 0$)

Right after the **resize()** (in either of the two cases) our array is half full

$$\frac{a.\text{length}}{2} - 1 \leq n \leq \frac{a.\text{length}}{2}$$

Suppose we have m calls to **add** (i, x) and **remove**(i). We want to show that the total time spent during all calls to **resize()** is $O(m)$.

Theorem 2.1 - proof

n is the size of the list **NOW**

Consider two consecutive **resize()** operations: **current** (j) and **previous** ($j - 1$).

There are no **resize()** operations in between. Therefore, the size of the array right after the ($j - 1$)-st **resize()** and just before the j -th **resize()** is the same.

array right after the ($j - 1$)-st **resize()**



THEN

$$\frac{a.length}{2} - 1 \leq n \leq \frac{a.length}{2}$$

array just before the j -th **resize()** (triggered by either add or remove operation):

1. **add** (i, x) and $a.length = n$



2. **remove** (i) and $a.length \geq 3n$

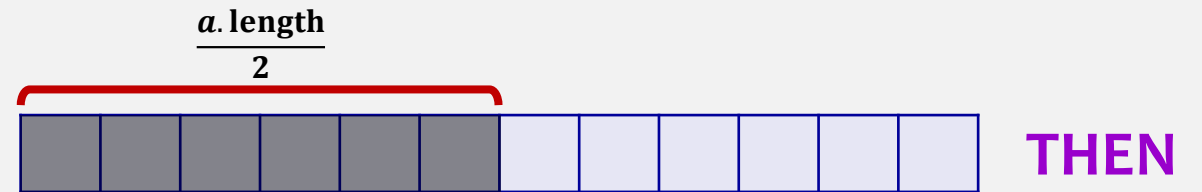


NOW

Theorem 2.1 - proof

n is the size of the list **NOW**

array right after the $(j - 1)$ -st **resize()**



array just before the j -th **resize()** (triggered by either add or remove operation):

1. **add** (i, x) and $a.length = n$



We added at least $(\# \text{ elements at } j\text{-th } \mathbf{resize}()) - (\# \text{ elements at } (j - 1)\text{-st } \mathbf{resize}())$

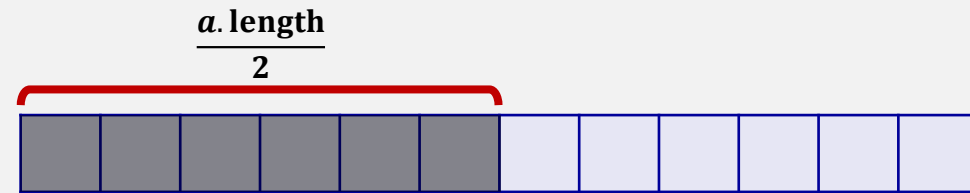
$$\text{We added} \geq a.length - \frac{a.length}{2} = \frac{a.length}{2} = \frac{n}{2}$$

We have at least $n/2$ **add** (i, x) operations between these two **resize()** operations (maybe more if we also have **remove** (i) operations)

Theorem 2.1 - proof

n is the size of the list **NOW**

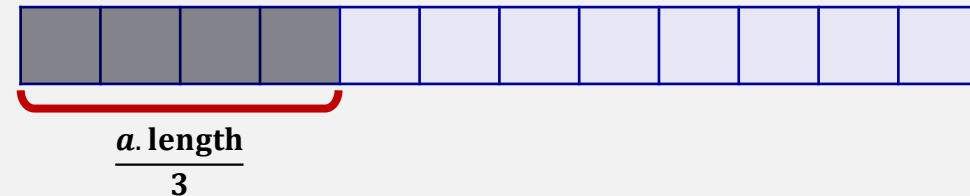
array right after the $(j - 1)$ -st **resize()**



THEN

array just before the j -th **resize()** (triggered by either add or remove operation):

2. **remove(i)** and $a.length \geq 3n$



NOW

We removed at least $(\# \text{ elements at } (j - 1)\text{-st } \mathbf{resize}()) - (\# \text{ elements at } j\text{-th } \mathbf{resize}())$

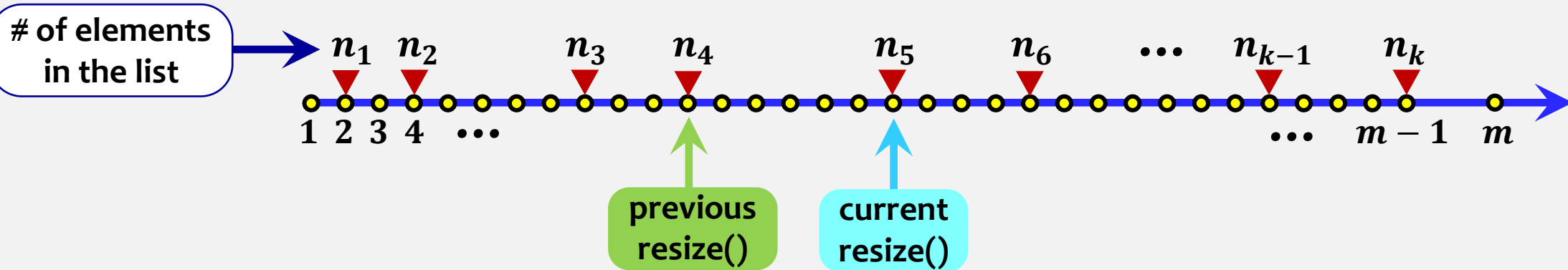
$$\text{We removed} \geq \frac{a.length}{2} - 1 - \frac{a.length}{3} = \frac{a.length}{6} - 1 \geq \frac{3n}{6} - 1 = \frac{n}{2} - 1$$

We have at least $n/2 - 1$ **remove(i)** operations between these two **resize()** operations (maybe more if we also have **add(i, x)** operations)

Theorem 2.1 - proof

▼ **resize()**

● add/remove operation



The total # of elements copied by all the calls to **resize()** is at most

$$n_1 + n_2 + n_3 + \dots + n_k$$

$$m \geq \frac{n_1}{2} + \frac{n_2}{2} + \frac{n_3}{2} + \dots + \frac{n_k}{2} = \frac{1}{2} \left(\begin{array}{l} \text{total \# of elements copied} \\ \text{by all the calls to \texttt{resize()}} \end{array} \right)$$

$$2m \geq \left(\begin{array}{l} \text{total \# of elements copied} \\ \text{by all the calls to \texttt{resize()}} \end{array} \right)$$

Total running time of all the calls to **resize()** is $O(m)$.

ArrayStack

The **ArrayStack** is an efficient way to implement a **Stack**.

In particular, we can implement:

- **push(x)** as **add(n, x)**
- **pop()** as **remove($n - 1$)**

in which case these operations will run in **$O(1)$** amortized time.

Why?

Recall:

add(i, x) and **remove(i)** run in **$O(1 + n - i)$** time per operation (ignoring **resize()**).